2nd report

문제 1. 퀵 정렬과 합병정렬의 성능을 측정하여 비교·분석하시오. 즉, 데이터 수 (충분히 큰) n을 변화시키면서 두 정렬 방법의 실행시간을 측정하여 비교하시오. 각 n에 대하여, 서로 다른 데이터를 여러 번 무작위로 생성하여 평균적인 실행시간을 구하고 비교하시오.

1. 프로그램 개요

합병정렬(Merge Sort)과 퀵 정렬(Quick Sort)의 성능을 비교하기 위해 만들어진 코드이다.
데이터 크기(배열의 크기)를 변화시키면서 두 정렬 알고리즘에 무작위 데이터를 여러 번 생성해서 평균 실행 시간을 알아내서 두 알고리즘의 차이를 비교하는 것을 목표로 한다.
코드에 필요한 조건은 무작위 데이터 생성 기능, 두 정렬이 같은 데이터를 처리하도록 구성하는 것, 정렬 과정과 시간 측정이 코드로 구현되는 것이다.

2. 프로그램 구조

헤더 파일로 stdio.h, time.h, stdlib.h를 포함하여 입출력, 시간 측정, 동적 메모리 할당과 같은 기능 구현을 구현하였다.
#define SWAP(x, y, t): 퀵정렬할 때 사용하기 위한 것으로 두 변수 x와 y의 값을 임시 변수 t를 사용하여 교환(swap)한다.
merge_sort_DC: 합병정렬을 재귀적으로 구현한 것으로, 배열의 시작(low)과 끝(high)을 기준으로 배열을 두 부분으로 나누고, 분할된 두 배열을 각각 정렬하기 위해 재귀적으로 merge_sort_DC를 호출한다. 두 배열이 정렬되면 merge 함수를 호출하여 합치고 최종적으로 정렬된 배열을 만든다.
merge: 합병정렬에서 두 부분 배열 병합을 위해서 필요한 함수이다.
배열의 중간 인덱스(mid)를 기준으로 왼쪽(low부터 mid)과 오른쪽(mid+1부터 high)을 나눈다. 두 배열의 크기를 계산하고 동적 메모리를 사용하여 임시 배열(sorted)을 생성한다. 동적 메모리를 사용한 이유는 임시 배열의 크기를 효율적으로 할당하기 위해서이다.
두 부분 배열의 첫 번째 요소를 비교하며 작은 값을 임시 배열에 저장하고 이 과정을 두 배열 중 하나가 끝날 때까지 병합을 계속 진행한다. 그리고 둘 중 하나가 끝나면 남아 있는 배열의 값들을 임시 배열에 추가한다. 마지막에는 임시 배열을 원본 배열(list)에 복사하고, 동적 메모리를 해제한다.
quick sort_DC: 배열을 피봇(pivot)을 기준으로 나누고, 왼쪽과 오른쪽 부분 배열을 재귀적으로 정렬하여 최종적으로 정렬된 배열을 생성한다.
배열의 시작(low)과 끝(high)을 기준으로 partition 함수를 호출하여 피봇을 기준으로 배열을 나눈다. 분할된 피벗의 왼쪽 부분 배열(low부터 pivot_pos - 1)과 오른쪽 부분 배열(pivot_pos + 1부터 high)을 각각 정렬하기 위해 재귀적으로 quicksort_DC를 호출한다.

partition: 피봇(pivot)을 기준으로 배열을 재구성하여 피벗보다 작은 값은 왼쪽, 큰 값은 오른

쪽으로 이동시킨다. 배열의 첫 번째 값을 피봇으로 선택하고, 두 개의 인덱스(i, j)를 사용하여 배열을 순회한다. (i는 피봇보다 큰 값을 찾는 인덱스, j는 피봇보다 작은 값을 찾는 인덱스)
i가 j보다 작을 경우, 두 인덱스의 값을 교환하고, 그렇지 않으면 피봇을 j 위치로 이동시켜 피봇의 올바른 위치를 설정한다. 그리고 마지막에 피벗의 최종 위치(j)를 반환하여 이후 재귀 호출에서 사용한다.
random: rand를 이용하여 랜덤 데이터를 배열에 초기화하였다. 랜덤 데이터의 범위는 배열 크기와 동일하게 설정했다.
main: 랜덤값을 받을 list 배열과 정렬에 사용할 배열 list_copy를 사용해서 list_copy 배열에 list 배열을 넣고 두 정렬에 각각 사용하였다. 이때 같은 랜덤 데이터를 사용하였다. 분석 및 비교를 위한 시간 측정을 위해 <time.h>라이브러리를 선언하였고, clock을 활용하여 시간을 측정하였다. 또한, 반복문을 활용하여 10번씩 결과를 측정하였다.

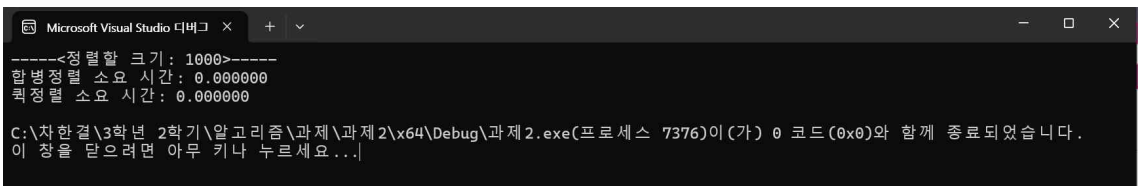
3. 프로그램 설계

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#define SWAP(x, y, t) ((t) = (x), (x) = (y), (y) = (t)) // x와 y의 값을 t를 이용해 교환
#define MAX_SIZE 100000// 배열의 크기
void merge_sort_DC(int list[], int low, int high);
void merge(int list[], int low, int mid, int high);
void quicksort_DC(int list[], int low, int high);
int partition(int list[], int low, int high);
void merge_sort_DC(int list[], int low, int high) { // 합병정렬
    int middle;
    if (low < high) { //부문제 크기가 2 이상
        middle = (low + high) / 2; // 중간 지점을 계산
        merge_sort_DC(list, low, middle); // 중간을 기준으로 왼쪽 정렬
        merge_sort_DC(list, middle + 1, high); // 중간을 기준으로 오른쪽 정렬
        merge(list, low, middle, high); //합치기
    }
}void merge(int list[], int low, int mid, int high) { //배열 합치기
    int n1 = mid - low + 1; // 왼쪽 배열의 크기
    int n2 = high - mid; // 오른쪽 배열의 크기
    int* sorted = (int*)malloc((n1 + n2) * sizeof(int)); //동적메모리
    int i = low, j = mid + 1, s = 0;
    while (i <= mid && j <= high) { //병합
        if (list[i] <= list[j]) {
            sorted[s++] = list[i++]; // 왼쪽 배열이 더 작으면 왼쪽 배열 값을 삽입
        }
        else {
```

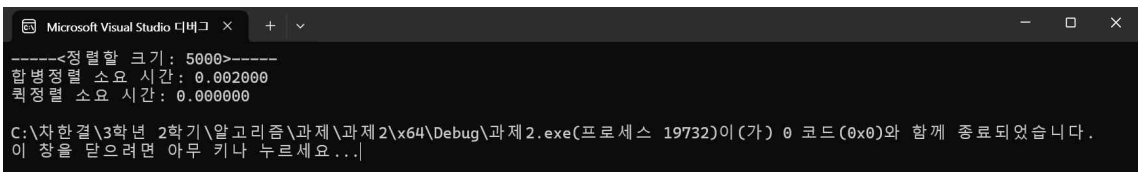
```
            sorted[s++] = list[j++]; // 오른쪽 배열이 더 작으면 오른쪽 배열 값을 삽입
        }
    }
    while (i <= mid) sorted[s++] = list[i++];  // 왼쪽 배열에 남은 값들 처리
    while (j <= high) sorted[s++] = list[j++]; // 오른쪽 배열에 남은 값들 처리
    for (int k = 0; k < (n1 + n2); k++) {  // 합쳐진 배열을 원본 배열에 복사
        list[low + k] = sorted[k];
    }
    free(sorted);  // 동적 메모리 해제
}void quicksort_DC(int list[], int low, int high) { // 퀵정렬
    int pivot_pos;
    if (low < high) { // 데이터가 하나 남을때까지
        pivot_pos = partition(list, low, high); // 분할
        quicksort_DC(list, low, pivot_pos - 1); // 왼쪽을 정렬
        quicksort_DC(list, pivot_pos + 1, high); // 오른쪽을 정렬
    }
}int partition(int list[], int low, int high) { //분할
    int pivot = list[low], i = low + 1, j = high, temp;
    while (i <= j) {
        while (i <= high && list[i] <= pivot) i++; // 피봇보다 작은 값 찾기
        while (j > low && list[j] > pivot) j--; // 피봇보다 큰 값 찾기
        if (i < j) {
            SWAP(list[i], list[j], temp); // i<j일경우 값을 교환
        }
    }
    SWAP(list[low], list[j], temp); // 피봇과 j 값을 교환하여 피봇을 올바른 위치로 보
냄
    return j; // 반환
}void random(int list[], int size) { //랜덤값을 배열에 넣음
    for (int i = 0; i < size; i++) {
        list[i] = rand() % 1000000; // 0부터 1000000사이 값을 랜덤으로 배열에 넣음
    }
}int main() {
    srand(time(NULL)); //난수 초기화
    int n = MAX_SIZE; // 배열의 크기
    int list[MAX_SIZE], list_copy[MAX_SIZE]; // 원본 배열과 정렬할 배열
    for (int i = 0; i < 10; i++)
    {
        random(list, n); // 배열에 랜덤값 넣기
        for (int i = 0; i < n; i++) list_copy[i] = list[i]; // 정렬할 배열에 원본 배열값
```

```
넣기
        printf("-----<정렬할 크기: %d>-----\n", n);
        clock_t start = clock(); //시간 측정 시작
        merge_sort_DC(list_copy, 0, n - 1); // 합병정렬
        clock_t end = clock(); //시간 측정 끝
        printf("합병정렬 소요 시간: %lf\n", (double)(end - start) /
CLOCKS_PER_SEC); //시간 출력
        for (int i = 0; i < n; i++) list_copy[i] = list[i]; // 정렬할 배열에 원본 배열값
넣기
        start = clock(); //시간 측정 시작
        quicksort_DC(list_copy, 0, n - 1); // 퀵정렬
        end = clock(); //시간 측정 끝
        printf("퀵정렬 소요 시간: %lf\n\n", (double)(end - start) /
CLOCKS_PER_SEC); //시간 출력
    }
    return 0; // 프로그램 종료
}
```

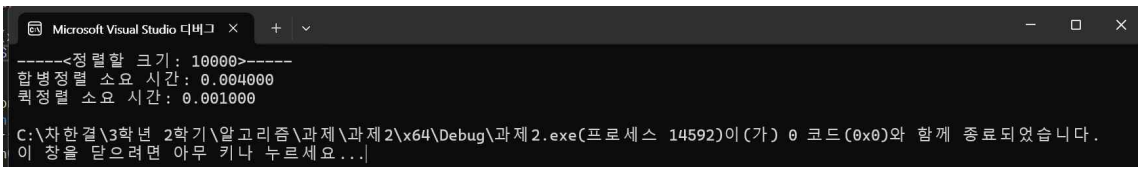
4. 프로그램 실행 결과



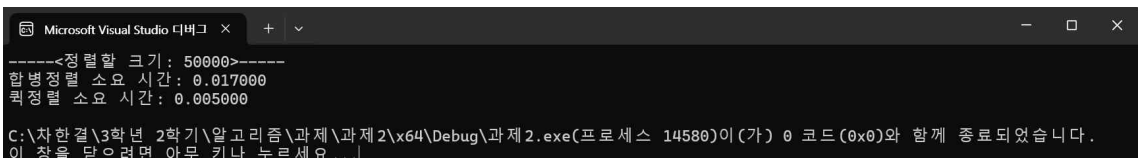
크기가 1000까지는 데이터가 너무 작아 둘 다 시간이 0으로 비교할 수 없다.



5000으로 크기를 설정했을 경우 합병정렬만 시간이 나온다.



10000으로 크기를 설정할 경우 합병정렬이 0.004, 퀵 정렬이 0.001로 합병정렬이 소요시간이 더 오래 걸리는 것을 알 수 있다.



50000으로 크기를 설정할 경우 합병정렬이 0.017, 퀵 정렬이 0.005로 여전히 합병정렬이 소요시간이 더 길다.

```
Microsoft Visual Studio 디버그
-----<정렬할 크기: 100000>-----
합병정렬 소요 시간: 0.037000
퀵정렬 소요 시간: 0.011000

C:\차한결\3학년 2학기\알고리즘\과제\과제2\x64\Debug\과제2.exe(프로세스 7920)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

```
Microsoft Visual Studio 디버그
-----<정렬할 크기: 110000>-----
합병정렬 소요 시간: 0.045000
퀵정렬 소요 시간: 0.012000

C:\차한결\3학년 2학기\알고리즘\과제\과제2\x64\Debug\과제2.exe(프로세스 17484)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

```
Microsoft Visual Studio 디버그
-----<정렬할 크기: 120000>-----
합병정렬 소요 시간: 0.041000
퀵정렬 소요 시간: 0.012000

C:\차한결\3학년 2학기\알고리즘\과제\과제2\x64\Debug\과제2.exe(프로세스 7836)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

크기를 100000으로 설정할 경우 합병정렬이 0.037, 퀵 정렬이 0.011로 합병정렬이 소요시간이 더 길다.

```
#define MAX_SIZE 130000
Microsoft Visual Studio 디버그

C:\차한결\3학년 2학기\알고리즘\과제\과제2\x64\Debug\과제2.exe(프로세스 16968)이(가) -1073741571 코드(0xc00000fd)와 함께
종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

13만부터는 배열의 크기가 너무 커져서 메모리 제한으로 인해 출력되지 않는다.

5. 결과 분석

1만 10번 출력

```
-----<정렬할 크기: 10000>-----
합병정렬 소요 시간: 0.004000
  퀵정렬 소요 시간: 0.001000

-----<정렬할 크기: 10000>-----
합병정렬 소요 시간: 0.003000
  퀵정렬 소요 시간: 0.001000

-----<정렬할 크기: 10000>-----
합병정렬 소요 시간: 0.003000
  퀵정렬 소요 시간: 0.001000

-----<정렬할 크기: 10000>-----
합병정렬 소요 시간: 0.003000
  퀵정렬 소요 시간: 0.001000

-----<정렬할 크기: 10000>-----
합병정렬 소요 시간: 0.003000
  퀵정렬 소요 시간: 0.000000

-----<정렬할 크기: 10000>-----
합병정렬 소요 시간: 0.003000
  퀵정렬 소요 시간: 0.001000

-----<정렬할 크기: 10000>-----
합병정렬 소요 시간: 0.003000
  퀵정렬 소요 시간: 0.001000

-----<정렬할 크기: 10000>-----
합병정렬 소요 시간: 0.003000
  퀵정렬 소요 시간: 0.001000

-----<정렬할 크기: 10000>-----
합병정렬 소요 시간: 0.003000
  퀵정렬 소요 시간: 0.001000

-----<정렬할 크기: 10000>-----
합병정렬 소요 시간: 0.004000
  퀵정렬 소요 시간: 0.000000


C:\차한결\3학년 2학기\알고리즘\과제\과제2\x64\Debug\과제2.exe(프로세스 9788)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

5만 10번 출력

```
Microsoft Visual Studio 디버그

-----<정렬할 크기: 50000>-----
합병정렬 소요 시간: 0.018000
  퀵정렬 소요 시간: 0.005000

-----<정렬할 크기: 50000>-----
합병정렬 소요 시간: 0.017000
  퀵정렬 소요 시간: 0.005000

-----<정렬할 크기: 50000>-----
합병정렬 소요 시간: 0.015000
  퀵정렬 소요 시간: 0.005000

-----<정렬할 크기: 50000>-----
합병정렬 소요 시간: 0.016000
  퀵정렬 소요 시간: 0.005000

-----<정렬할 크기: 50000>-----
합병정렬 소요 시간: 0.015000
  퀵정렬 소요 시간: 0.004000

-----<정렬할 크기: 50000>-----
합병정렬 소요 시간: 0.016000
  퀵정렬 소요 시간: 0.005000

-----<정렬할 크기: 50000>-----
합병정렬 소요 시간: 0.015000
  퀵정렬 소요 시간: 0.004000

-----<정렬할 크기: 50000>-----
합병정렬 소요 시간: 0.016000
  퀵정렬 소요 시간: 0.004000

-----<정렬할 크기: 50000>-----
합병정렬 소요 시간: 0.014000
  퀵정렬 소요 시간: 0.005000

-----<정렬할 크기: 50000>-----
합병정렬 소요 시간: 0.015000
  퀵정렬 소요 시간: 0.005000


C:\차한결\3학년 2학기\알고리즘\과제\과제2\x64\Debug\과제2.exe(프로세스 20408)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

10만 10번

```
Microsoft Visual Studio 디버그

-----<정렬할 크기: 100000>-----
합병정렬 소요 시간: 0.034000
  퀵정렬 소요 시간: 0.010000

-----<정렬할 크기: 100000>-----
합병정렬 소요 시간: 0.029000
  퀵정렬 소요 시간: 0.010000

-----<정렬할 크기: 100000>-----
합병정렬 소요 시간: 0.030000
  퀵정렬 소요 시간: 0.009000

-----<정렬할 크기: 100000>-----
합병정렬 소요 시간: 0.029000
  퀵정렬 소요 시간: 0.009000

-----<정렬할 크기: 100000>-----
합병정렬 소요 시간: 0.031000
  퀵정렬 소요 시간: 0.011000

-----<정렬할 크기: 100000>-----
합병정렬 소요 시간: 0.029000
  퀵정렬 소요 시간: 0.010000

-----<정렬할 크기: 100000>-----
합병정렬 소요 시간: 0.029000
  퀵정렬 소요 시간: 0.010000

-----<정렬할 크기: 100000>-----
합병정렬 소요 시간: 0.029000
  퀵정렬 소요 시간: 0.009000

-----<정렬할 크기: 100000>-----
합병정렬 소요 시간: 0.029000
  퀵정렬 소요 시간: 0.009000

-----<정렬할 크기: 100000>-----
합병정렬 소요 시간: 0.029000
  퀵정렬 소요 시간: 0.010000


C:\차한결\3학년 2학기\알고리즘\과제\과제2\x64\Debug\과제2.exe(프로세스 20880)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

프로그램 실행결과에서는 정렬의 크기에 따라 시간 측정이 가능한지를 확인한 후 1만, 5만,

10만이 시간 측정에 적합하다고 판단하여, 합병정렬과 퀵 정렬 과정에서 for 문을 사용해서 10번씩 반복하여 1만, 5만, 10만 결과를 출력하였다.

배열 크기	합병정렬 평균시간(초)	퀵 정렬 평균시간(초)
1만	0.0032	0.0008
5만	0.0157	0.0047
10만	0.298	0.088

표를 보면 데이터의 크기가 작을 때는 두 정렬의 성능 차이가 작지만, 데이터 크기가 커질수록 퀵 정렬이 더 빠르다는 것을 알 수 있다.

문제 2. (충분히 큰) n개의 데이터에 대하여, 퀵 정렬의 임곗값을 변화시키면서 성능을 측정하여 그 결과를 분석하시오(임계값 이하의 데이터는 삽입 정렬을 사용).
1) 각 임계값 별로, 서로 다른 데이터를 여러 번 무작위로 생성하여 평균적인 실행시간을 구하고 비교하시오.
2) 현재 임곗값이 t라면, 쪼개진 부문제 크기가 t보다 클 동안에는 퀵 정렬을 사용하다가 부분제 크기가 t 이하로 줄어들면 삽입 정렬(교재 6장 참조) 사용
3) 임곗값을 0부터 n까지 (0이면 퀵 정렬만 사용, n이면 삽입 정렬만 사용) 변화시키면서 측정
4) 분석 결과에는 최적의 임계값에 대한 내용이 포함되어야 함

1. 프로그램 개요
이 프로그램은 퀵 정렬과 삽입 정렬을 결합하여, 임계값에 따른 정렬 성능을 비교하고 최적의 임계값을 도출하는 실험을 수행한다. 임계값 t는 문제를 퀵 정렬로 재귀적으로 나누는 기준점이고, t 이하의 크기를 가진 배열은 삽입 정렬로 처리하고 t 이상의 크기를 가진 배열은 퀵 정렬로 처리한다. 랜덤 데이터를 여러 번 생성하고, 임계값(t)을 0에서 n까지 변화시키면서 측정하여 각 t에 대한 평균 실행 시간을 비교한 후, 최적의 t 값을 찾아낸다.

2. 프로그램 구조
헤더 파일로 stdio.h, time.h, stdlib.h를 포함하여 입출력, 시간 측정, 동적 메모리 할당과 같은 기능 구현을 구현하였다.
#define SWAP(x, y, t): 퀵정렬할 때 사용하기 위한 것으로 두 변수 x와 y의 값을 임시 변수 t를 사용하여 교환(swap)한다.
quick sort_DC: 1번의 퀵 정렬과 유사하지만, 퀵 정렬과 삽입 정렬을 혼합한 방식을 사용한다. 따라서 인자로 threshold를 추가하였다. 임계값이 0인 경우 high - low + 1 <= threshold 조건이 거짓이 되기 때문에 퀵 정렬만 사용하게 되고, 임계값이 n인 경우 high - low + 1 <= threshold 조건이 참이 되기 때문에 삽입 정렬을 사용하게 된다. 나머지 경우에는 배열 크기가 threshold 이하인 경우 삽입 정렬이 사용되고, threshold를 초과하면 퀵 정렬이 사용된다. 배열을 피봇(pivot)을 기준으로 나누고, 왼쪽과 오른쪽 부분 배열을 재귀적으로 정렬하여 최종적으로 정렬된 배열을 생성한다. 배열의 시작(low)과 끝(high)을 기준으로 partition 함수를 호출하여 피봇을 기준으로 배열을 나눈다. 분할된 피벗의 왼쪽 부분 배열(low부터 pivot_pos - 1)과 오른쪽 부분 배열(pivot_pos + 1부터 high)을 각각 정렬하기 위해 재귀적으로 quicksort_DC를 호출한다.

partition: 피봇(pivot)을 기준으로 배열을 재구성하여 피벗보다 작은 값은 왼쪽, 큰 값은 오른쪽으로 이동시킨다. 배열의 첫 번째 값을 피봇으로 선택하고, 두 개의 인덱스(i, j)를 사용하여 배열을 순회한다. (i는 피봇보다 큰 값을 찾는 인덱스, j는 피봇보다 작은 값을 찾는 인덱스)
i가 j보다 작을 경우, 두 인덱스의 값을 교환하고, 그렇지 않으면 피봇을 j 위치로 이동시켜 피봇의 올바른 위치를 설정한다. 그리고 마지막에 피벗의 최종 위치(j)를 반환하여 이후 재귀 호출에서 사용한다.
insertion_sort: 반복문에서 i는 정렬되지 않은 배열의 첫 번째 원소부터 시작하고, 정렬할 원소는 next 변수에 저장한다. while 문을 통해 정렬된 부분의 끝에서부터 next보다 큰 원소를 찾는다. list[j]가 next보다 크면, 해당 값을 오른쪽으로 한 칸 이동하고, 적절한 위치를 찾으면, next를 list[j + 1]에 삽입한다. 이러한 과정을 정렬이 완성될 때까지 반복한다.
random: rand를 이용하여 랜덤 데이터를 배열에 초기화하였다. 랜덤 데이터의 범위는 배열 크기와 동일하게 설정했다.
main: 분석 및 비교를 위한 시간 측정을 위해 <time.h>라이브러리를 선언하였고, clock을 활용하여 시간을 측정하였다. 랜덤 데이터를 저장하는 배열로 list[MAX_SIZE]를 사용하였고, 다양한 임계값을 저장하는 배열로 thresholds[]를 사용하고, 평균 실행 시간을 계산하기 위해 실험을 반복할 횟수(100회)를 num_trials에 저장하였다. 이중 for문을 사용해서 임계값을 변화하면서 100번씩 시간을 측정하여 평균을 구한 다음 임계값과 시간을 출력하였다.

3. 프로그램 설계

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h> // 시간 측정을 위해서 필요한 라이브러리
#define SWAP(x, y, t) ((t) = (x), (x) = (y), (y) = (t)) // 두 변수의 값을 교환한다.
#define MAX_SIZE 100000 // 배열 크기
void quicksort_DC(int list[], int low, int high, int threshold);
int partition(int list[], int low, int high);
void insertion_sort(int list[], int low, int high);
void random(int list[], int size);
void quicksort_DC(int list[], int low, int high, int threshold) { //퀵정렬
    if (high - low + 1 <= threshold) { // 정렬할 크기가 임계값 이하인지 확인
        insertion_sort(list, low, high); // 삽입 정렬
    }
    else { // 정렬할 크기가 임계값보다 큰 경우
        int pivot_pos = partition(list, low, high); //분할
        quicksort_DC(list, low, pivot_pos - 1, threshold); // 왼쪽을 정렬
        quicksort_DC(list, pivot_pos + 1, high, threshold); // 오른쪽을 정렬
    }
}int partition(int list[], int low, int high) { //분할
    int pivot = list[low], i = low + 1, j = high, temp;
    while (i <= j) {
```

```
        while (i <= high && list[i] <= pivot) i++; // 피봇보다 작은 값 찾기
        while (j > low && list[j] > pivot) j--; // 피봇보다 큰 값 찾기
        if (i < j) {
            SWAP(list[i], list[j], temp); // i<j일경우 값을 교환
        }
    }
    SWAP(list[low], list[j], temp); // 피봇과 j 값을 교환하여 피봇을 올바른 위치로 보
냄
    return j; // 반환
}void insertion_sort(int list[], int low, int high) {//삽입정렬
    for (int i = low + 1; i <= high; i++) { // 두번째 원소부터 순서대로 처리
        int next = list[i]; //삽입할 값을 저장
        int j = i - 1; // 이전 원소를 비교하기 위해서 초기화
        while (j >= low && list[j] > next) { // 삽입 위치를 찾기 위해 반복
            list[j + 1] = list[j]; //한 칸씩 뒤로 이동
            j--;
        }
        list[j + 1] = next; // 삽입 위치에 값 저장
    }
}void random(int list[], int size) { //랜덤값을 배열에 넣는다.
    for (int i = 0; i < size; i++) {
        list[i] = rand() % MAX_SIZE; // 0부터 MAX_SIZE 사이 값을 랜덤 생성
    }
}int main() {
    srand(time(NULL)); // 난수 초기화
    int list[MAX_SIZE]; //정렬할 배열
    int thresholds[] = {0,10,100,1000,10000}; //실험할 임계값들 설정
    int num_thresholds = sizeof(thresholds) / sizeof(thresholds[0]); // 임계값 개수
계산
    int num_trials = 100; // 평균 계산을 위해서 반복할 횟수
    for (int x = 0; x < num_thresholds; x++) { // 각 임계값에 대해 반복
        int threshold = thresholds[x]; //임계값 설정
        double total_time = 0.0; //시간 초기화
        for (int y = 0; y < num_trials; y++) { // 반복 횟수만큼 반복
            random(list, MAX_SIZE); //랜덤값 생성
            clock_t start = clock(); // 시간 측정 시작
            quicksort_DC(list, 0, MAX_SIZE - 1, threshold); //퀵정렬
            clock_t end = clock(); // 시간 측정 끝
            total_time += (double)(end - start) / CLOCKS_PER_SEC; //시간을 누적
        }
```

```
        printf("임계값: %d, 시간: %lf(s)\n", threshold, total_time / num_trials); //
평균 시간 출력
    }
    return 0;
}
```

4. 프로그램 실행 결과

```
Microsoft Visual Studio 디버그
임계값: 0, 시간: 0.023660(s)
임계값: 10, 시간: 0.020060(s)
임계값: 100, 시간: 0.020720(s)
임계값: 1000, 시간: 0.067170(s)
임계값: 10000, 시간: 0.574190(s)

C:\차한결\3학년 2학기\알고리즘\과제\과제2-2\x64\Debug\과제2-2.exe(프로세스 7408)이(가) 0 코드(0x0)와 함께 종료되었습니다
.
이 창을 닫으려면 아무 키나 누르세요...
```

```
Microsoft Visual Studio 디버그
임계값: 0, 시간: 0.023300(s)
임계값: 10, 시간: 0.020050(s)
임계값: 20, 시간: 0.019790(s)
임계값: 30, 시간: 0.019360(s)
임계값: 40, 시간: 0.018770(s)
임계값: 50, 시간: 0.019340(s)
임계값: 60, 시간: 0.019360(s)
임계값: 70, 시간: 0.019650(s)
임계값: 80, 시간: 0.019740(s)
임계값: 90, 시간: 0.020240(s)
임계값: 100, 시간: 0.020380(s)

C:\차한결\3학년 2학기\알고리즘\과제\과제2-2\x64\Debug\과제2-2.exe(프로세스 11844)이(가) 0 코드(0x0)와 함께 종료되었습니
다.
이 창을 닫으려면 아무 키나 누르세요...
```

```
Microsoft Visual Studio 디버그
임계값: 20, 시간: 0.019590(s)
임계값: 30, 시간: 0.019110(s)
임계값: 40, 시간: 0.018840(s)
임계값: 50, 시간: 0.019070(s)
임계값: 60, 시간: 0.019230(s)
임계값: 70, 시간: 0.019310(s)

C:\차한결\3학년 2학기\알고리즘\과제\과제2-2\x64\Debug\과제2-2.exe(프로세스 12640)이(가) 0 코드(0x0)와 함께 종료되었습니
다.
이 창을 닫으려면 아무 키나 누르세요...
```

```
Microsoft Visual Studio 디버그
임계값: 30, 시간: 0.019980(s)
임계값: 31, 시간: 0.019570(s)
임계값: 32, 시간: 0.019340(s)
임계값: 33, 시간: 0.019490(s)
임계값: 34, 시간: 0.019250(s)
임계값: 35, 시간: 0.019450(s)
임계값: 36, 시간: 0.019210(s)
임계값: 37, 시간: 0.019090(s)
임계값: 38, 시간: 0.019380(s)
임계값: 39, 시간: 0.019480(s)
임계값: 40, 시간: 0.019460(s)
임계값: 41, 시간: 0.019470(s)
임계값: 42, 시간: 0.019610(s)
임계값: 43, 시간: 0.019340(s)
임계값: 44, 시간: 0.019100(s)
임계값: 45, 시간: 0.019200(s)
임계값: 46, 시간: 0.019180(s)
임계값: 47, 시간: 0.019470(s)
임계값: 48, 시간: 0.019440(s)
임계값: 49, 시간: 0.019290(s)
임계값: 50, 시간: 0.019310(s)

C:\차한결\3학년 2학기\알고리즘\과제\과제2-2\x64\Debug\과제2-2.exe(프로세스 16820)이(가) 0 코드(0x0)와 함께 종료되었습니
다.
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용
하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

5. 결과 분석

배열 크기는 10만을 기준으로 하고 100번 반복하여 평균을 구해서 결괏값을 출력하였다. 처음에는 0부터 10000까지 임계값을 두고 출력하였고, 10일 때 임계값이 가장 작게 나와서 0~100까지로 범위를 설정하여 출력하고, 50일 때 임계값이 가장 작게 나오고 그 다음으로 30과 60일 때 작게 나와서 20~70까지 다시 범위를 바꿔서 측정하였다. 20~70까지 범위를 했을 때 40일 때 가장 작게 나와서 30~50 사이로 범위를 바꿔서 측정하였다. 30~50 사이로 측정했을 때 시간이 가장 작게 나온 것은 임계값이 37일때로 나왔다. 시간이 적게 나올수록 임계값이 최적인 상태이기 때문에 삽입정렬과 퀵정렬이 가장 효율적으로 작동한 임계값 37이 가장 최적의 임계값이다. 퀵 정렬만 사용한 임계값 t=0일때는 실행 시간이 비교적 느리게 나오고, 삽입정렬만 사용한 임계값 t=1000 일때는 실행 시간이 크게 증가했다. 퀵정렬만 사용한 경우나 삽입정렬만 사용한 경우보다 둘 다 혼합해서 사용한 경우가 더 효율적인 것을 알 수 있고, 퀵 정렬과 삽입 정렬의 조합은 배열 크기에 따라 성능이 크게 달라진다는 것을 알 수 있다.

참고문헌

차근차근 이해하는 알고리즘 -저자 이형원 3장 합병정렬, 퀵정렬 파트, 6장 삽입정렬, 퀵정렬 파트 / 데이터구조 에센스 -저자 이형원 13장 퀵정렬 파트